

Paralelização do Algoritmo Q-Learning com OpenMP

Rayan Raddatz de Matos (00584919)¹, Eduardo Magnus Lazuta (00325091)¹,
Enzo Lisbôa Peixoto (00584827)¹

¹ Instituto de Informática, Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

1. Algoritmo e Problema

O Q-Learning é um método de aprendizado por reforço que treina um agente por meio de episódios. Em cada episódio, o agente parte do estado inicial e executa ações até atingir o objetivo ou esgotar o limite de passos. Cada passo atualiza a Q-Table com a equação de Bellman: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$. Após o treinamento, a política ótima é extraída como $\pi(s) = \arg \max_a Q(s, a)$.

O problema escolhido é pathfinding em grade $L \times C$: o agente parte de $(0, 0)$ e deve atingir $(L-1, C-1)$ desviando de obstáculos com as ações cima, baixo, esquerda, direita. Algoritmo 1 descrever o comportamento completo da aplicação sequencial.

2. Paralelização

2.1. Escolha do Nível de Paralelismo

Quando paramos para discutir como paralelizar, tivemos três ideias:

- **Ações** (seleção do argmax): descartada, pois o número de ações é apenas 4, o overhead de sincronização seria muito maior que qualquer ganho.
- **Steps** (passos dentro de um episódio): descartada, pois cada step depende do estado resultante do passo anterior, dependência de dados verdadeira faria com que as threads ficassem esperando e trabalhando sequencialmente por baixo dos panos.
- **Episódios**: escolhida. Episódios são independentes entre si: cada um parte do estado inicial, executa uma trajetória completa e atualiza a Q-Table. Não há dependência entre episódios distintos, o que permite distribuição direta via `omp for`.

Após decidir começamos a paralelizar com o `omp parallel for`, entretanto após algumas execuções de teste vimos que apenas paralelizar não seria o suficiente. Quando executamos vimos que nunca havia convergência para o algoritmo paralelizado apesar de ter um grande speedup. Isso acontecia pois ao rodar um problema com 10000 episódios e 4 threads, por exemplo, cada thread executava 2500 episódios, mas acabava por não comunicar seu conhecimento com as outras threads, então o conhecimento final das 4 threads era o mesmo que rodar sequencialmente com 2500 episódios, pensando nesse problema tivemos que implementar sincronização ao código, limitando o potencial paralelo.

2.2. Sincronização no treino

Cada thread mantém uma Q-Table local e executa um lote de episódios de forma autônoma. Ao final de cada lote (*chunk* de `sync_interval` episódios) as tabelas são fundidas na Q-Table global e redistribuídas. O ciclo repete até esgotar os episódios:

Algorithm 1 Q-Learning Sequencial

Require: Grid(L, C), Obstáculos(O), Episódios(E), Max_Passos(P)

Require: α (Aprendizado), γ (Desconto), ε (Exploração)

1. INICIALIZAÇÃO E AMBIENTE

- 1: Alocar dinamicamente $Q_Table[Estados][Ações]$
- 2: Preencher Q_Table com zeros
- 3: Gerar Grid(L, C) com Estado_Inicial s_0 em $(0, 0)$, Estado_Objetivo g em $(L-1, C-1)$
- 4: e posicionar O obstáculos aleatórios (evitando s_0 e g)
- 5: Configurar Recompensas: $R_{objetivo}(+100)$, $R_{obstáculo}(-100)$, $R_{passo}(-1)$, $R_{loop}(-10)$

2. NÚCLEO DE APRENDIZADO (Treinamento)

- 6: **for** cada episódio de 1 até E **do**
- 7: $s \leftarrow s_0$ (Estado Inicial)
- 8: **for** cada passo de 1 até P **do**
- 9: *// Seleção de Ação (ε -Greedy)*
- 10: **if** Random(0, 1) < ε **then**
- 11: $a \leftarrow$ Escolher_Ação_Aleatória()
- 12: **else**
- 13: $a \leftarrow \arg \max_{a'} Q[s][a']$ (Melhor ação conhecida)
- 14: **end if**
- 15: *// Interação com o Ambiente*
- 16: $s' \leftarrow$ Calcular_Próximo_Estado(s, a)
- 17: $r \leftarrow$ Avaliar_Recompensa(s')
- 18: **if** s' já foi visitado neste episódio **then**
- 19: $r \leftarrow r + R_{loop}$ ▷ Penalidade de ciclo
- 20: **end if**
- 21: *// Atualização de Bellman (Aprendizado Temporal)*
- 22: Max_Q_Futuro $\leftarrow$ Máximo($Q[s']$ [todas_ações])
- 23: $Q[s][a] \leftarrow Q[s][a] + \alpha \cdot (r + \gamma \cdot \text{Max_Q_Futuro} - Q[s][a])$
- 24: *// Transição de Estado*
- 25: $s \leftarrow s'$
- 26: **if** $s = g$ **then**
- 27: Encerrar Episódio (Objetivo Alcançado)
- 28: **end if**
- 29: **end for**
- 30: **end for**

3. PÓS-PROCESSAMENTO E SAÍDA

- 31: Política_Ótima $\pi(s) \leftarrow \arg \max_a Q[s][a]$
 - 32: Imprimir Visualização do Grid com $\pi(s)$
 - 33: Simular Caminho Final do Agente
 - 34: Liberar Memória Alocada
-

Listing 1. Regiao paralela de train com sincronizacao (parallel_qlearning_cli.c)

```
1 #pragma omp parallel private(episode, state, action)
2 {
3     int tid = omp_get_thread_num();
4     QLearning local_ql = *ql;
5     local_ql.q_table = local_q_tables[tid]; /* Q-Table propria por
6         thread */
7     local_ql.rand_seed = cfg->seed + (unsigned int)tid;
8     /* copia tabela global para local antes de comecar */
9     #pragma omp barrier
10
11     while (episode_start < cfg->num_episodes) {
12         /* Treinamento local -- omp for distribui o chunk */
13         #pragma omp for schedule(runtime)
14         for (episode = episode_start; episode < episode_end; episode++)
15             run_episode(&local_ql, episode);
16         /* barreira implicita apos omp for */
17
18         /* Merge com delta na Q-Table global */
19         int active = (chunk_size < num_threads) ? chunk_size :
20             num_threads;
21         #pragma omp for collapse(2)
22         for (state = 0; state < ql->num_states; state++)
23             for (action = 0; action < NUM_ACTIONS; action++) {
24                 double snapshot = ql->q_table[state][action];
25                 double delta_sum = 0.0;
26                 for (tt = 0; tt < num_threads; tt++)
27                     delta_sum += local_q_tables[tt][state][action] -
28                         snapshot;
29                 ql->q_table[state][action] = snapshot + delta_sum /
30                     active;
31             }
32
33         /* Broadcast -- global copiada de volta para local */
34         for (state ...) for (action ...)
35             local_ql.q_table[state][action] = ql->q_table[state][action];
36         #pragma omp barrier
37
38         episode_start = episode_end;
39     }
40 }
```

Por que delta e não média absoluta? Se um chunk tem menos episódios que threads, algumas threads ficam ociosas e mantêm `local_ql.q_table` igual ao broadcast anterior (delta zero). Usar a média absoluta dividiria a soma dos updates por T mesmo com apenas $k < T$ threads ativas, diluindo o aprendizado. O merge por delta divide apenas pelas k threads ativas ($\text{active} = \min(\text{chunk_size}, T)$), preservando o α efetivo de cada update.

Heurísticas de `sync_interval`: foram implementadas duas estratégias automáticas no dispatcher `train()`:

Listing 2. Calculo automatico do `sync_interval` (train())

```

1 if (strcmp(cfg->sync_mode, "sqrt") == 0)
2     /* sqrt(N/T): equilibra overhead de barreira e divergencia local */
3     sync_interval = (int)sqrt((double)cfg->num_episodes / (double)
4         num_threads);
5 else
6     /* statespace: cada thread explora todos os estados antes de
7     sincronizar */
8     sync_interval = ql->num_states;    /* = L * C */

```

A heurística *sqrt* é inspirada em limites de *regret* para SGD paralelo: intervalos maiores aumentam a divergência entre tabelas locais; menores aumentam o overhead das barreiras. A heurística *statespace* parte da intuição de que cada thread deve visitar (em média) todos os estados do grid uma vez antes de fundir o conhecimento, promovendo exploração mais diversa. Além disso, testamos duas sincronizações fixas: a cada 10 episódios e 1000 episódios.

2.3. HOGWILD!

Ao nos depararmos com o problema da sincronização, também buscamos se existiam formas de fugir dela, nisso encontramos o algoritmo HOGWILD! [Niu et al. 2011] . Ele elimina toda sincronização, nesse caso todas as threads leem e escrevem na mesma Q-Table sem sincronização, aceitando *race conditions* intencionalmente. A implementação é minimal:

Listing 3. train_hogwild com Q-Table compartilhada

```

1 #pragma omp parallel private(episode)
2 {
3     int tid = omp_get_thread_num();
4
5     /* Copia local apenas para rand_seed independente por thread.
6     * importante: q_table herda o ponteiro da struct original ---
7     * todas as threads leem/escrevem na mesma Q-Table sem nenhuma
8     sincronizacao ou lock. */
9     QLearning local_ql = *ql;
10    local_ql.rand_seed = cfg->seed + (unsigned int)tid * 1000003U;
11
12    #pragma omp for schedule(runtime)
13    for (episode = 0; episode < cfg->num_episodes; episode++)
14        run_episode(&local_ql, episode);    /* le/escreve shared sem
15        lock */

```

o ponteiro é copiado por valor, continuando a apontar para a Q-Table original compartilhada. Apenas *rand_seed* é isolado por thread, garantindo trajetórias distintas sem overhead de sincronização.

Três propriedades do problema justificam a correte prática do método:

- **Esparsidade:** cada episódio visita uma fração pequena dos $L \times C$ estados; conflitos simultâneos em $Q(s, a)$ são raros.
- **Escala dos updates:** cada atualização é ponderada por $\alpha \leq 0,1$; sobrescrever um update ocasional é um ruído de ordem $O(\alpha)$, desprezível para a convergência.

- **Simetria direcional:** ambos os valores concorrentes são gradientes válidos calculados de trajetórias distintas; nenhum é incorreto.

Importante notar que nesse caso, quanto mais esparsa a entrada, no nosso caso, quanto mais estados existirem, melhor performará o algoritmo.

3. Metodologia Experimental

O experimento adota um design fatorial completo sobre os fatores listados na Tabela 1. Para cada combinação de parâmetros, o tempo de treinamento (medido dentro de `train()`) é registrado em múltiplos blocos de replicação, permitindo estimar a variância de tempo devida ao escalonador do sistema operacional.

Table 1. Fatores e níveis do experimento.

Fator	Níveis	Motivação
Threads	1, 2, 4, 8, 16, 20, 40	Cobre desde serial até hyperthreading completo
Tamanho do grid	10, 50, 100	Espaço de estados pequeno, médio e grande
Modo de sync	<code>sqrt</code> , <code>statespace</code> , <code>hogwild</code>	Três estratégias de paralelização
Episódios	5000, 10000	Limiar de convergência: 5k instável, 10k estável
Densidade obst.	0%, 50%	Grid livre vs. grade com alta densidade
Schedule OpenMP	<code>static</code> , <code>guided</code> , <code>dynamic</code>	Políticas de distribuição de trabalho
Chunk	0, 1, 10	Granularidade do chunking para modo <code>guided/dynamic</code>

3.1. Baseline Sequencial

Linhas com `threads=1` invocam o binário sequencial puro, sem runtime OpenMP. Isso garante que o speedup seja calculado contra o tempo real do algoritmo, sem incluir o overhead de criação do pool de threads do OpenMP no denominador.

3.2. Afinidade de Threads

Importante notar que adotamos a política de afinidade de acordo com o número de threads:

- $T \leq 20$: `OMP_PLACES=cores`, `OMP_PROC_BIND=close`, uma thread por núcleo físico, sem hyperthreading, minimizando contenção de cache.
- $T = 40$: `OMP_PLACES=threads`, `OMP_PROC_BIND=close`, uma thread por CPU lógica, utilizando hyperthreading completo.

3.3. Medição de Tempo

O tempo registrado em `training_time_s` corresponde exclusivamente à execução de `train()`, excluindo inicialização do ambiente, alocação de memória e impressão de resultados. O binário paralelo usa `omp_get_wtime()` e o sequencial usa `clock_gettime(CLOCK_MONOTONIC)`, ambos medindo tempo de parede (*wall-clock*). Também foi pega o tempo do VTUNE, entretanto não foi utilizado por demonstrar uma variabilidade muito maior.

4. Resultados

4.1. Speedup e Eficiência

O speedup médio por número de threads e modo de sincronização está apresentado na Figura 1. O eixo tracejado representa o speedup ideal (linear). Para grids 10x10, o speedup satura rapidamente em torno de 4–5x a partir de 8 threads, indicando que o trabalho por thread é insuficiente para amortizar o overhead de sincronização. Para grids maiores (100x100), o HOGWILD! atinge speedup médio de até 17x com 40 threads, enquanto os modos com sincronização (*sqr*t, *statespace*) ficam abaixo de 10x. A diferença se deve ao fato de que os modos sincronizados impõem barreiras periódicas que serializam parcialmente a execução.

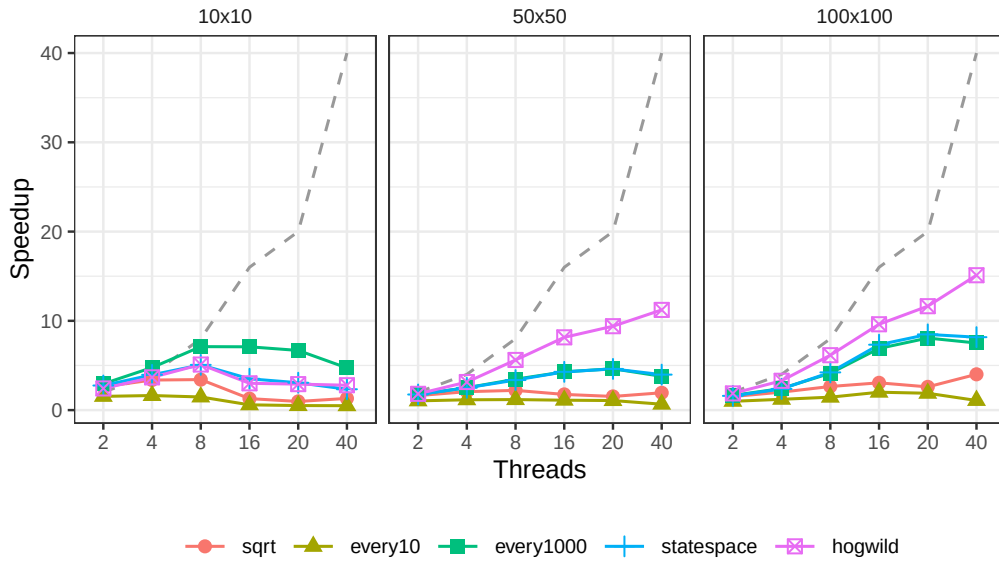


Figure 1. Speedup médio por modo de sincronização (10k episódios). Tracejado = ideal linear.

A eficiência paralela ($S(T)/T$) é mostrada na Figura 2. No grid 10x10, vários modos apresentam eficiência acima de 100%, possivelmente por efeitos de cache com múltiplos núcleos acessando partições distintas da Q-Table. Ainda assim, a eficiência colapsa rapidamente: nenhuma configuração sustenta 50% além de 8 threads neste grid, resultado consistente com o tráfego de coerência de cache discutido na Seção de Análise de Cache.

Para grids maiores, o HOGWILD! mantém a melhor eficiência em todos os tamanhos: cerca de 35–45% com 40 threads no grid 100x100, enquanto os demais modos ficam

abaixo de 15%. Entre os modos sincronizados, o *every10* (sincronização a cada 10 episódios) apresenta a pior eficiência. O overhead de barreiras frequentes supera o ganho da convergência mais rápida. O *every1000* e o *statespace* mantêm eficiência intermediária, pois sincronizam com menos frequência e amortizam o custo das barreiras sobre mais trabalho útil.

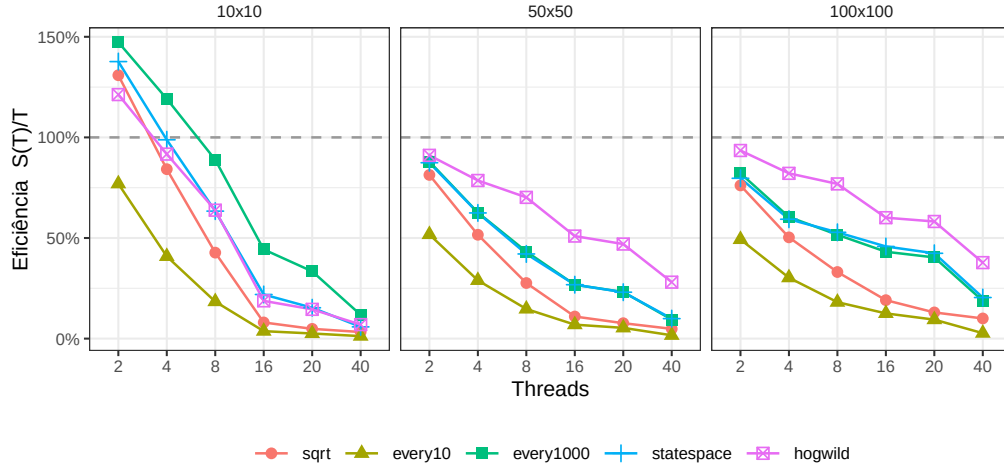


Figure 2. Eficiência paralela por modo de sincronização (10k episódios). Ideal = 100% (tracejado).

4.2. Análise de Cache (10x10)

Uma coisa que procuramos entender mais a fundo foi a perda do speedup no grid 10x10 após 8 threads. Para isso usamos o `perf stat` (eventos `L1-dcache-load-misses`, `LLC-loads` e `LLC-load-misses`) para todas as combinações de sync-mode e schedule com 10k episódios pois o vtune apresentou erros de versão. A Q-Table do grid 10x10 ocupa apenas 3,2 KB (100 estados $\times$ 4 ações $\times$ 8 bytes), cabendo inteiramente em L1 (32 KB). O baseline sequencial confirma isso com apenas 0,18% de L1-miss.

Table 2. L1-miss e LLC-miss por threads e modo de sync (media dos tres schedules, grid 10x10, 10k ep.).

Threads	sq-L1	sq-LLC	spc-L1	spc-LLC	hog-L1	hog-LLC
2	1,12%	0,80%	1,10%	1,11%	7,42%	0,44%
4	3,87%	0,70%	4,10%	0,79%	8,53%	0,18%
8	6,37%	0,17%	5,15%	0,18%	23,20%	0,11%
16	3,95%	30,9%	4,64%	23,1%	5,99%	36,7%
20	5,08%	29,1%	11,09%	22,9%	1,38%	36,4%
40	1,99%	40,7%	1,98%	28,5%	4,17%	35,9%

As máquinas hype tem um dual-socket (2x Intel Xeon E5-2650 v3, 10 núcleos por socket). Com `OMP_PLACES=cores` e `OMP_PROC_BIND=close`, os threads são alocados em núcleos consecutivos: $T \leq 10$ ficam inteiramente no socket~0; $T = 16$ já usa 10 núcleos do socket~0 e 6 do socket~1.

Para $T \leq 8$, hogwild apresenta L1-miss elevado (7–23%) mas LLC-miss abaixo de 0,5% (Tabela 2): cada escrita na Q-Table compartilhada invalida a linha de cache nos demais núcleos, forçando recargas de L2/L3, mas os dados são encontrados antes do LLC. Os modos sincronizados (sqrt, statespace) mantêm L1-miss baixo ($\leq 6,4\%$) pois cada thread opera em cópia local sem invalidações.

A partir de $T = 16$, threads do socket 1 passam a acessar a Q-Table alocada na memória do socket 0 para o hogwild. O que aparece como LLC-miss no L3 local do socket 1. Para o hogwild esse custo é pago em cada acesso, pois não há cópia local: daí os 36,7% de LLC-miss. Para statespace, os threads do socket 1 carregam a cópia local uma vez e a reutilizam por 100 episódios antes do próximo merge, amortizando os misses e chegando a apenas 23,1%.

Em todos os modos, a fronteira entre socket 0 e socket 1 é o ponto de ruptura do LLC-miss, confirmando que o gargalo do grid 10x10 com muitas threads é tráfego de cache, não escassez de trabalho por thread.

4.3. Análise de Cache (100x100)

A Q-Table do grid 100x100 ocupa 320 KB (10^4 estados $\times$ 4 ações $\times$ 8 bytes), excedendo o L1 (32 KB) e o L2 (≈ 256 KB) mas cabendo no LLC (20 MB). O baseline sequencial já exibe 2,82% de L1-miss (vs. 0,18% no grid 10x10), confirmando que a tabela transborda o L1 desde o início.

Table 3. L1-miss e LLC-miss por threads e modo de sync (media dos tres schedules, grid 100x100, 10k ep.).

Threads	sq-L1	sq-LLC	ev1k-L1	ev1k-LLC	spc-L1	spc-LLC	hog-L1	hog-LLC
2	5,63%	0,06%	3,39%	0,08%	3,28%	0,08%	4,50%	0,00%
4	6,27%	0,12%	2,91%	0,29%	2,62%	0,05%	4,30%	0,00%
8	7,80%	0,01%	2,64%	0,04%	2,09%	0,05%	5,41%	0,20%
16	11,79%	7,01%	1,98%	3,12%	2,18%	1,59%	4,36%	35,44%
20	12,85%	22,43%	2,83%	14,03%	2,17%	5,43%	3,78%	33,49%
40	11,03%	27,65%	3,01%	16,47%	2,01%	2,61%	2,56%	60,77%

Para $T \leq 8$, o LLC-miss permanece abaixo de 0,3% em todos os modos (Tabela 3): a Q-Table inteira cabe no LLC e cada thread opera em sua cópia local sem competição inter-socket. A partir de $T = 16$ os comportamentos divergem:

- **Hogwild** atinge 35,4% de LLC-miss com 16 threads e 60,8% com 40. A única Q-Table de 320 KB é escrita continuamente por todos os threads, invalidando linhas.
- **Statespace** chega a apenas 2,6% com 40 threads. O intervalo de sincronização é igual ao número de estados (10^4 episódios para o grid 100x100), resultando em uma única sincronização durante o treino.
- **Every1000** (16,5% com 40 threads) fica entre sqrt e statespace: sincroniza a cada 1000 episódios, menos frequente que o heurístico sqrt (27,7%).

No eixo de L1-miss, every10 (não incluído na tabela) alcança 10–15% em todos os níveis de threads por copiar os 320 KB da Q-Table a cada 10 episódios, expulsando dados úteis do L1 repetidamente. Os demais modos ficam entre 2–13%, reflexo direto de quão frequentemente o merge copia a tabela inteira.

Diferente do grid 10x10 onde o L1-miss é desprezível e o gargalo aparece de forma abrupta no LLC a partir de $T = 16$, no grid 100x100 o L1-miss já é estrutural e o LLC-miss reflete diretamente a frequência de sincronização: intervalos curtos geram mais tráfego de coerência, intervalos longos preservam a localidade.

4.4. Análise VTune

O Intel VTune Profiler foi utilizado no modo *performance-snapshot* para coletar IPC (*Instructions Per Cycle*) e percentual de *Memory Bound* por configuração (Figura 3).

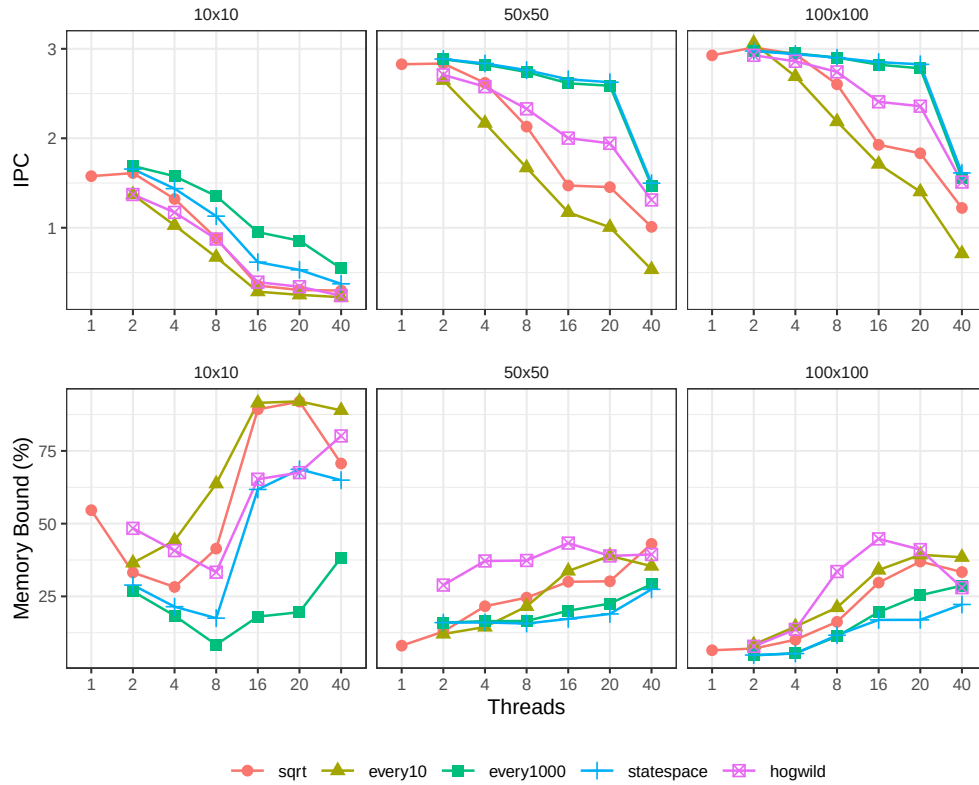


Figure 3. IPC (topo) e Memory Bound % (baixo) por modo de sincronização e tamanho de grid.

IPC decai com o número de threads em todos os cenários. Isso é esperado pois IPC mede eficiência de execução por thread, e mais threads aumentam a contenção por cache e barramento sem aumentar o throughput individual. Além disso, ao ativar o hyperthreading, dois threads lógicos compartilham o mesmo núcleo físico, pressionando ainda mais a cache. O conjunto de linhas de cache ativas dobra sem aumentar a capacidade do L1. Ainda no tópico do grid 10x10, vemos um alto Memory Bound e baixo IPC, reforçando o que foi visto na análise das cache. Esse tamanho faz com que múltiplas threads atualizem um conjunto muito restrito de linhas de cache com alta frequência.

4.5. Análise de fatores de menor impacto

Além do modo de sincronização, o experimento variou o schedule do OpenMP (static, dynamic, guided), o tamanho de chunk (auto, 1, 10) e a densidade de obstáculos (0%, 50%).

Os efeitos desses fatores concretos no tempo de execução e speedup não foi significativo. Os efeitos são analisados abaixo para o grid 100x100 com 10k episódios.

Schedule e chunk. A Figura 4 mostra o speedup médio por schedule, separado por tamanho de chunk e grid. Os três schedules produzem speedup praticamente idêntico: as durações de episódio são homogêneas e qualquer distribuição de carga é equilibrada. A diferença geral permanece pequena (<2% no speedup médio), indicando que o schedule não é o fator dominante. O chunk tem impacto igualmente pequeno.

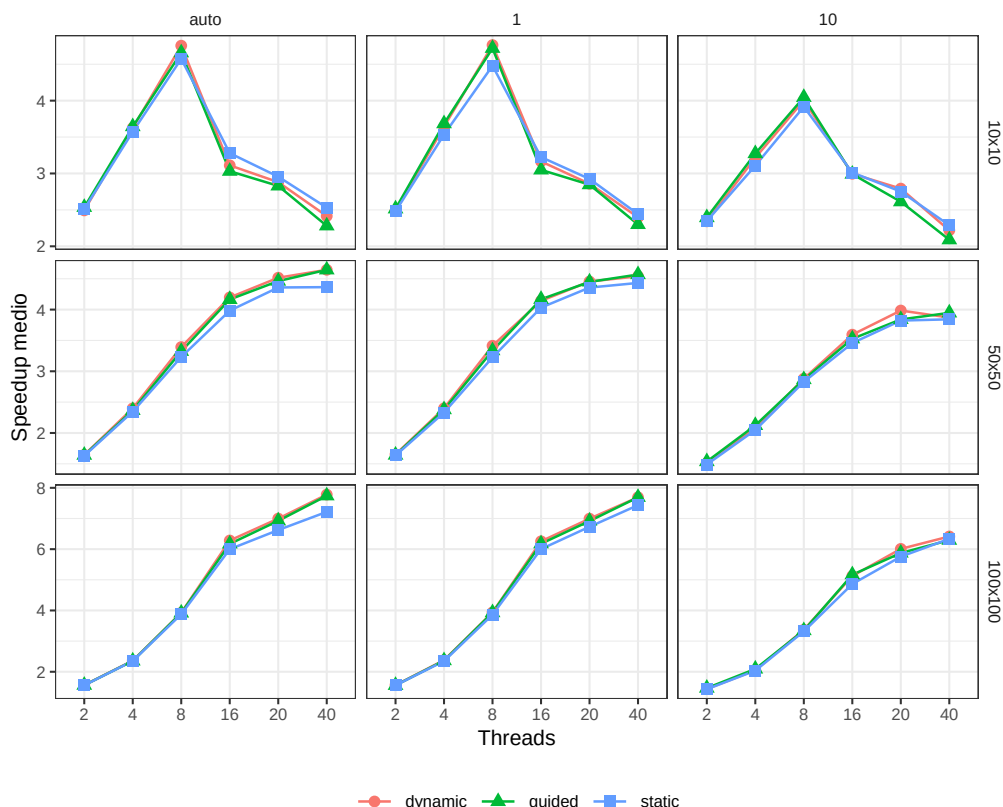


Figure 4. Speedup medio por schedule, separado por chunk e grid (10k ep., media sobre obstacles e sync-mode).

Densidade de obstáculos. A Figura 5 mostra o tempo de treinamento absoluto para 0% e 50% de obstáculos. Com 50% de obstáculos, o agente nunca encontra o objetivo e cada episódio percorre todos os `max_steps` passos, tornando o tempo sequencial muito maior. É importante notar que comparar speedup entre as duas densidades seria injusto: o ganho aparente com 50% de obstáculos reflete apenas que há mais trabalho total a dividir entre as threads — não que o algoritmo paralelo seja mais eficiente. O gráfico de tempo absoluto deixa claro que ambas as versões (0% e 50%) escalam de forma semelhante com mais threads, mas partem de baselines completamente diferentes.

4.6. Convergência

A Figura 6 apresenta a taxa de convergência em função do número de threads, separada por densidade de obstáculos, para o grid 100x100 com 10k episódios. Com 0% de obstáculos, todos os modos convergem razoavelmente para poucos threads, mas degradam

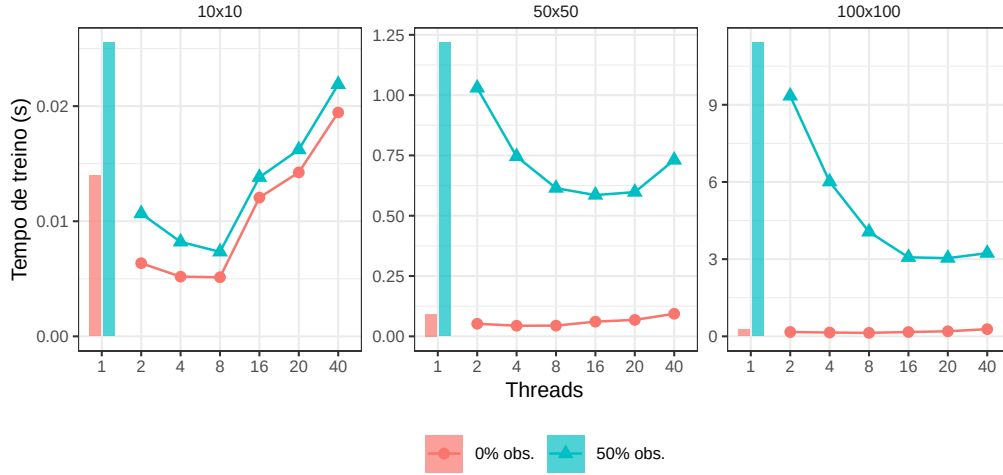


Figure 5. Tempo de treino por densidade de obstáculos (10k ep., media sobre configs e schedules). Barra = sequencial (1 thread), linhas = paralelo.

com o aumento do paralelismo. Com 50% de obstáculos, com exceção do pequeno grid 10x10, nenhum dos outros convergiu.

A causa da degradação nos modos sincronizados é a {divisão de episódios}: com T threads e E episódios totais, cada thread treina com apenas E/T episódios. Quando E/T cai abaixo do limiar mínimo de convergência $E_{\min}$ do problema, o agente não acumula experiência suficiente para aprender uma política válida. Esse limiar é mais alto com 50% de obstáculos, pois o agente precisa explorar trajetórias mais longas até encontrar o objetivo, exigindo mais episódios para propagar valores Q corretos a todos os estados relevantes.

Mesmo o HOGWILD! usando uma Q -Table compartilhada, que em teoria acumula atualizações de *todos* os threads simultaneamente não foi capaz de convergir nesses casos com muitos obstáculos. Isso nos diz que apenas uma boa heurística não é o suficiente para o problema convergir, também é necessário aumentar o tamanho do problema.

4.7. Speedup das Configurações Convergadas

A tabela a seguir apresenta o speedup médio considerando apenas as execuções que convergiram (10k episódios). Células vazias indicam que nenhuma execução da combinação convergiu.

5. Conclusão

Este trabalho paralelizou o Q -Learning por episódios usando OpenMP com diferentes estratégias e heurísticas.

O principal limitante de convergência identificado foi a divisão de episódios. com T threads e E episódios fixos, cada thread treina com apenas E/T episódios. Quando esse valor cai abaixo do limiar mínimo do problema, o agente não aprende uma política válida. A sincronização e as heurísticas selecionadas conseguem fazer o cenário ficar favorável, mas ainda não garantem precisão. Com 50% de obstáculos, nenhuma configuração paralela convergiu para mapas "grandes", incluindo o HOGWILD!, que embora acumule

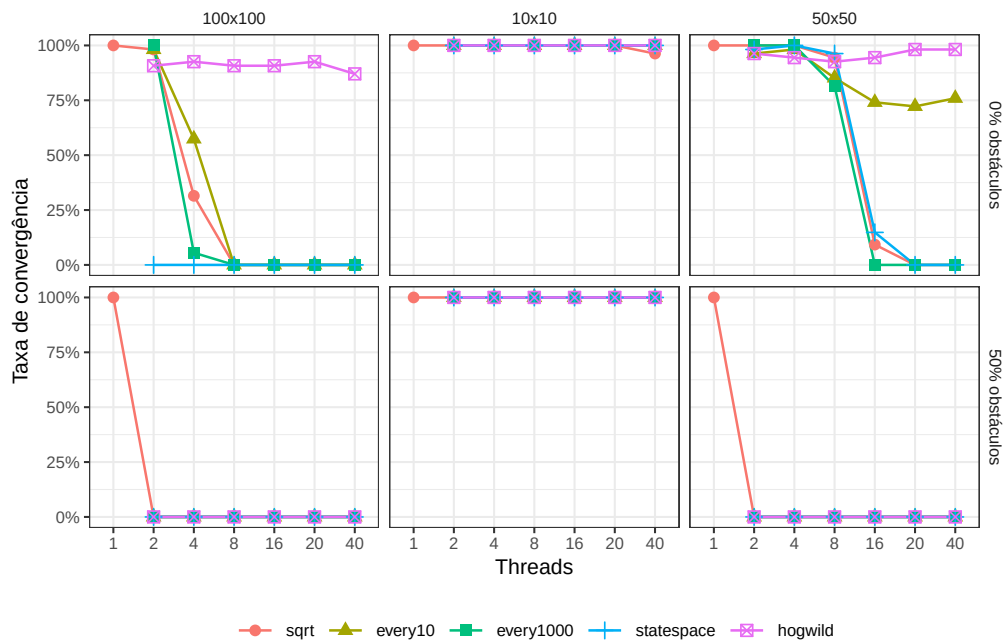


Figure 6. Taxa de convergência por modo de sincronização e densidade de obstáculos (10k ep.).

**Table 4. Speedup medio apenas para execucoes convergidas (10k episodios).
Celulas vazias: nenhuma execucao convergiu nessa combinacao.**

Grid	Config	2	4	8	16	20	40
10x10	sqrt	2.62	3.37	3.42	1.29	0.97	1.32
10x10	every10	1.54	1.63	1.47	0.59	0.51	0.49
10x10	every1000	2.95	4.77	7.11	7.09	6.69	4.73
10x10	statespace	2.75	3.95	5.07	3.51	3.07	2.34
10x10	hogwild	2.43	3.67	5.11	3.00	2.91	2.78
50x50	sqrt	1.98	2.33	2.20	1.19	NA	NA
50x50	every10	1.08	1.11	0.98	0.65	0.59	0.38
50x50	every1000	2.21	2.96	3.76	NA	NA	NA
50x50	statespace	2.20	2.91	3.58	4.16	NA	NA
50x50	hogwild	2.32	4.04	7.40	9.19	10.12	10.65
100x100	sqrt	1.77	1.99	NA	NA	NA	NA
100x100	every10	0.90	0.85	NA	NA	NA	NA
100x100	every1000	1.97	2.49	NA	NA	NA	NA
100x100	hogwild	2.43	4.24	8.00	11.45	13.88	18.14

updates de todas as threads, sofre com race conditions que corrompem o aprendizado em ambientes densos.

Em grids pequenos (10x10), o gargalo é diferente: a Q-Table cabe em L1, mas com grande quantidade de threads, ocorrem invalidações de coerência de cache que saturam o LLC e degradam IPC e speedup independentemente do modo de sincronização.

O HOGWILD! obteve o maior speedup bruto (até 17x com 40 threads em grids grandes), beneficiando-se da ausência de barreiras e do acesso esparsa à Q-Table em grids 100x100. Os modos sincronizados com intervalos longos (every1000, statespace) mostraram melhor comportamento de cache por manter Q-Tables locais isoladas por mais tempo entre syncs, mas demonstrarão pior conversão.

O benefício prático do paralelismo está na redução do tempo de treinamento para um orçamento fixo de episódios, em cenários onde esse orçamento dividido é suficiente para o tamanho e complexidade do problema.

References

Niu, F., Recht, B., Re, C., and Wright, S. (2011). Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. In *Advances in Neural Information Processing Systems*, volume 24.